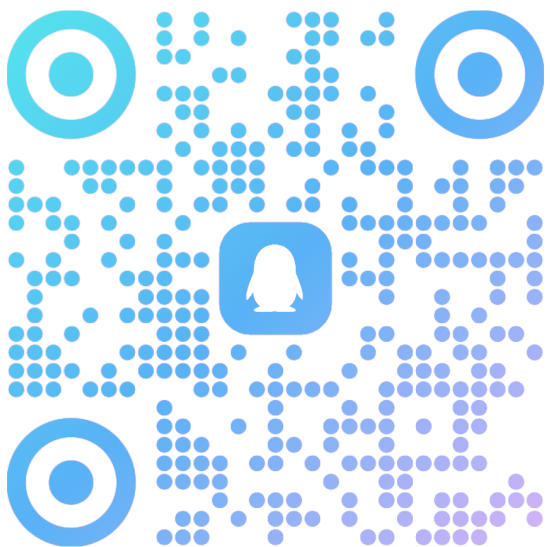




福州大学-2025编译...

群号: 1037351079



扫一扫二维码，加入群聊



编译原理

2025 春

任课教师: 柯嵩宇

2025-03-04

作业

- 提交要求
 - 格式：PDF
 - 语言：中文
 - 建议使用LaTeX完成，使用XeLaTeX编译
 - 发送到任课老师邮箱：songyuke@fzu.edu.cn
 - 文件和邮件命名规则：
 - 主题：2025编译原理课程作业1-<学号>-姓名
 - 附件名：2025编译原理课程作业1-<学号>-姓名.pdf
 - 邮件正文：没啥要求，记得礼貌用语和署名（姓名+时间）
- 允许**适度使用**大模型
 - 如使用大模型需提供额外信息：
 - 1. 大模型名称、版本、运营平台等；
 - 2. 提供完整的对话流程；
 - 作者本人应当对生成的内容负责
 - 对于只有大模型生成的内容，没有个人思考的作业会被扣除至少20%的“懒惰分”

注意邮件主题！ ！ ！

来信分类规则



编译原理学生作业1

如果: 主题 包含 2025编译原理课程作业1 (不区分大小写)

则: 移动到文件夹 作业1

且: 标记为 已读

且: 自动回复, 内容为: 作业1已收到, 本邮件系自动回复



2025编译原理课程作业2

如果: 主题 包含 2025编译原理课程作业2 (不区分大小写)

则: 移动到文件夹 作业2

且: 标记为 已读

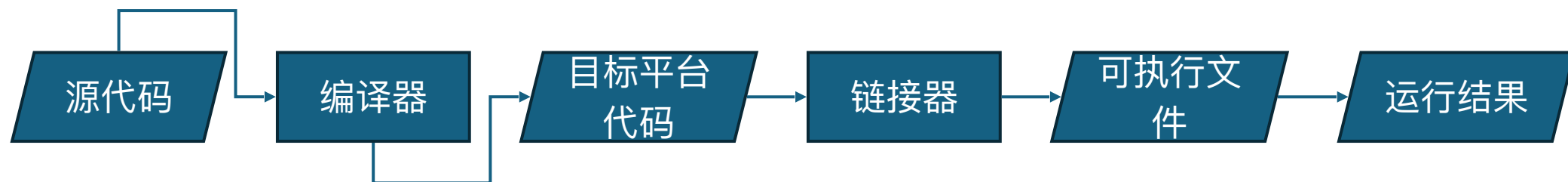
且: 自动回复, 内容为: 作业2已收到, 本邮件为自动回复

作业1

1. 使用C语言（不要用C++，太复杂了）解决一个需要使用二叉查找树（红黑树、AVL等）的OJ题目（可以从你们历史代码里面找）；
2. 使用GCC编译器，分别用O1、O2、O3三个级别的优化开关进行编译，在汇编代码层面比较优化的效果
3. 使用Clang-LLVM编译器，添加--emit-llvm参数，在LLVM IR层面比较O1、O2、O3的优化效果
4. 撰写一个图文并茂的报告

课前复习

计算机程序的执行流程



代表语言： C/C++、 Pascal、 Rust、 Go

编译程序的主要工作阶段

- 可执行文件(Executable File)的生成通常包含下列步骤：
 - 预处理(Preprocessing, gcc -E)
 - 宏展开、条件编译、文件包含
 - 编译(Compilation, gcc -S)
 - 词法分析、语法分析、语义分析、中间代码生成、机器无关优化、机器相关优化、目标代码生成
 - 汇编(Assemble, gcc -c)
 - 二进制文件生成
 - 链接(Linking)
 - 填充符号表

扩展阅读：《程序员的自我修养——链接、装载与库》

现代编译器的编译流程

- 词法分析
 - 输入：字符流 (char stream) ，即源代码
 - 输出：词元流 (token stream)
 - 描述方法：正则表达式 (正则文法)
 - 处理工具：有限状态自动机
- 语法分析
 - 输入：词元流
 - 输出：语法树 (Syntax Tree)
 - 描述方法：上下文无关文法
 - 处理工具：下推自动机 (抽象形式，实际处理LL或者LR方法)

编译流程

- 语义分析（语义检查）(对应教材的语法制导翻译器)
 - 输入：（具体）语法树
 - 输出：（抽象）语法树
 - 目的：
 - 1.检查程序逻辑，符合语言定义，没有二义性；
 - 2. 添加必要中间过程（如推断数据类型、添加隐式类型转换等）
 - 例子：字符串+数值、整数加减浮点数

编译流程

- 中间表示生成 (Intermediate Representation)
 - 输入: AST
 - 输出: IR
 - 目的: 去除计算机执行无关的语义信息, 比如类的访问控制
- 平台无关优化
 - 输入: IR
 - 输出: Optimized IR
 - 目的: 常量压缩、常量函数求值 (constexpr)、消除无用代码、函数内联等
- 平台相关代码优化
 - 输入: IR
 - 输出: Optimized IR
 - 目的: 展开循环、分配寄存器
- 代码生成
 - 输入: IR
 - 输出: 汇编指令、字节码等
 - 目的: 指令选择、SIMD优化等

带大家过一遍教材

大家拿出书或者打开PDF版的目录

作业2

- 《编译原理》 P78 3.3节练习
 - 3.3.1 (i)(ii)(iii)X(1)(2)(5)(7)
 - 3.3.2 全部
 - 文字或数学形式描述，建议后者，列出可以明显看出规律的集合元素
 - 3.3.3
 - 简答
 - 3.3.4
 - 简答

第二讲 程序语言基础

编程语言的构成

语法

- 声明
- 定义
- 语句

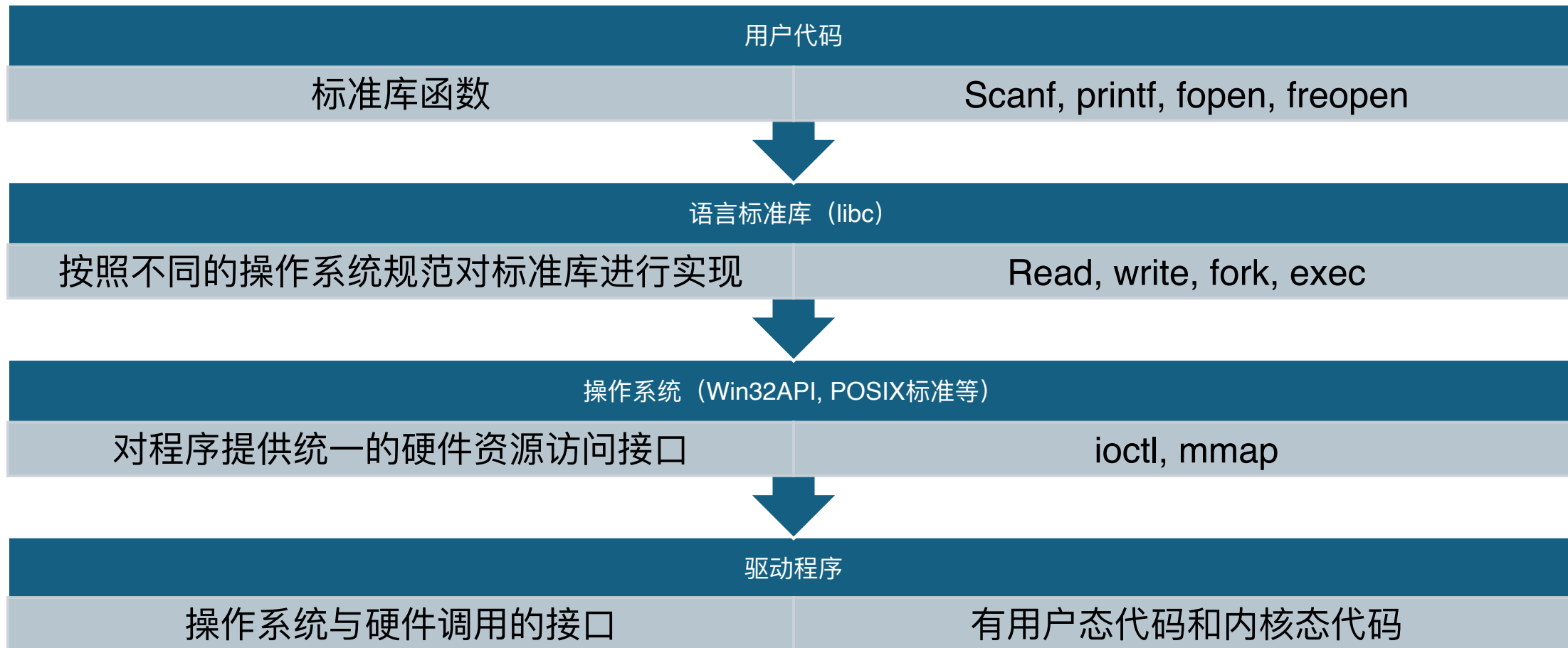
语义

- 顺序、分支、循环
- 面向对象
- 函数式编程
- 类型转换

语言标准库

- 输入输出函数
- 访问文件的接口
- 内存申请、释放
- 常用数据结构,
list、*map*
- 信息安全相关函数
- 网络访问支持

代码、标准库、POSIX接口、系统内核



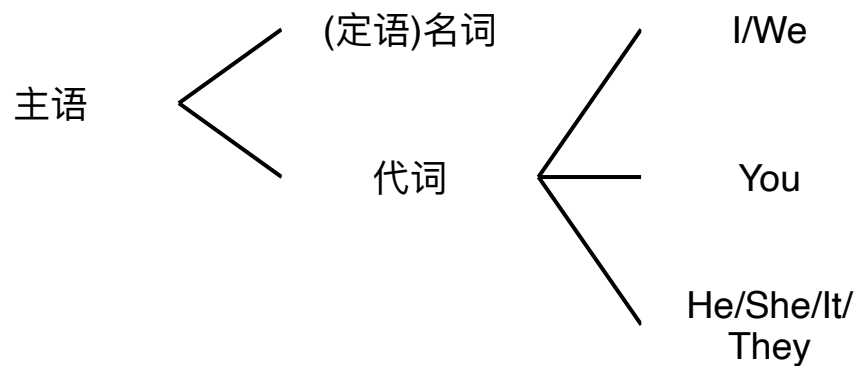
什么是语言？

- 语言(language)是一个记号系统，完整的定义包括文法(grammar)和语义(semantics)
- 文法是语言的结构规则系统，其主要功能是指导语言的生成和理解，确保句子在结构上是合法的。
- 语义研究语言符号所表达的意义和意义之间的关系。

文法怎么定义？

- 一个句子怎么构成？

I	write	a book.
主语	谓语	宾语



文法怎么定义？

- 一个句子怎么构成？

I	write	a book.
主语	谓语	宾语

谓语 — 动词（状语）

文法怎么定义？

- 一个句子怎么构成？

I	write	a book.
主语	谓语	宾语

如何表示文法？

- 数学定义

- $G = (V_N, V_T, S, P)$
- V_N 是非终结符号集合
- V_T 是终结符号集合，可以理解为最终字符，其中有一个特殊符号 ε ，用于表示空
- S 是初始符号
- P 是产生式的集合

- 产生式/推导式

$$A \rightarrow B \mid C \mid D \mid E \mid \dots$$

文法的分类（乔姆斯基分类法）

- 0型文法，无限制文法
 - 对推导式左右均没有限制 $\alpha B \rightarrow A\gamma$
 - 自然语言至少是第0类文法

文法的分类（乔姆斯基分类法）

- 1类文法——上下文相关文法

- 出现在左侧的非终结符一定出现在右侧

$$\alpha S \beta \rightarrow \alpha \gamma \beta$$

$$\alpha S \gamma \rightarrow \alpha \beta \gamma$$

$$S \rightarrow \alpha \alpha \beta \beta$$

- 有的文献要求左侧长度（符号数量）不得长于右侧

文法的分类（乔姆斯基分类法）

- 2类文法——上下文无关文法（Context-free Grammar, CFG）

- 终结符号只能出现在生成式的右边

$$S \rightarrow AB$$

$$A \rightarrow HE$$

$$H \rightarrow \delta$$

$$E \rightarrow \tau$$

$$B \rightarrow HSE$$

文法的分类（乔姆斯基分类法）

- 3类文法——正则文法（Regular Grammar, RG）
 - 终结符号如果有则只能出现在生成式的右边，且只能是右侧的前缀

$$S \rightarrow \alpha B$$

$$B \rightarrow \beta H$$

$$H \rightarrow \sigma A$$

$$A \rightarrow HS$$

词法分析的核心

正则表达式与非确定有限状态自动机

正则文法 $\Leftrightarrow$ 正则表达式

- 无前导0的正整数

$[1-9][0-9]^*$

- 浮点数

$\left(\left([1-9][0-9]^* \right) \mid 0 \right) ? . [0-9]^*$

- 十六进制整数

$0x[0-9A-Fa-f]^*$

极简正则表达式

- 只有两个字符0和1
- 0^* 表示0个或多个连续的0
- 0^+ 表示1个或多个连续的0
- 0^5 表示5个连续的0，等价于00000
- $0|1$ 表示识别一个0或者1
- $000|101$ 表示识别连续的000或者101
- $00(0|1)01$ 表示识别00001或者00101，括号是用来表示子匹配模式或者叫改变运算优先级的

正则语言

- 一个正则表达式能够识别的所有字符串的集合被称作一种正则语言

$$0^*1 = \{1, 01, 001, 0001, 00001, \dots\}$$

$$0^*1^* = \{0, 1, 01, 001, 011, 0111, 0011, \dots\}$$

$$(0 \mid 1)^* = ?$$

$$1^*00 = ?$$

$$? = \{\epsilon, 01, 0011, 000111, 00001111, \dots\}$$

有穷状态自动机 (Finite Automation)

- FA是一个字符串处理工具，它记录一个当前状态，每接收到一个字符，它就会根据当前状态和接收到的字符在状态间进行转移。
- 数学定义
 - 有穷状态自动机，也称有限状态自动机，可以被定义成一个五元组 $(\Sigma, S, s_0, \delta, F)$
其中， Σ 是字符集(alphabet)， S 是状态集合(states)， s_0 是初始状态(initial state)， δ 是状态转移函数(transition function)， F 是可接受状态(accepting state)的集合。
 - 状态转移函数 δ 的定义如下
$$\delta(s, \sigma) = t \in S$$
 - 可接受状态 $f \in S$ ，是一种特殊的状态。可以大致理解为，如果自动机处理完了所有输入后停留在可接受状态，它就会返回True

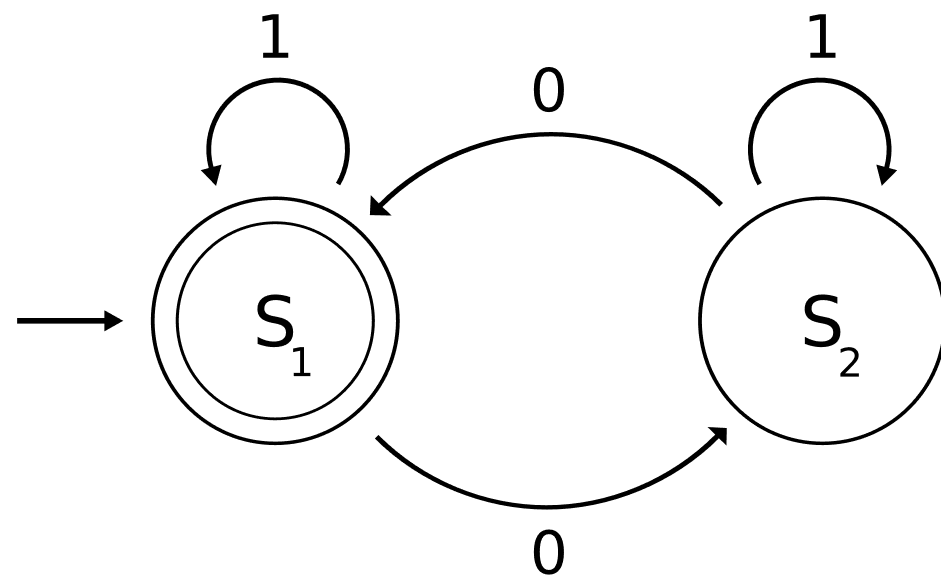
有穷状态自动机 (Finite Automation)

- 状态转移函数 δ 的定义如下

$$\delta(s, \sigma) = t \in S$$

- 可接受状态 $f \in S$ ，是一种特殊的状态。可以大致理解为，如果自动机处理完了所有输入后停留在可接受状态，它就会返回True
 - *Once all input has been received, if the **current** state is **an accepting state**, the input is accepted; otherwise it is rejected.*

示例



非确定有穷状态自动机

(Nondeterministic Finite Automation)

- 数学定义

- 非确定有穷状态自动机可以被定义成一个五元组

$$(\Sigma, S, s_0, \delta, F)$$

其中, Σ 是字符集(alphabet), S 是状态集合(states), s_0 是初始状态(initial state), δ 是状态转移函数(transition function), F 是可接受状态(accepting state)的集合。

- 状态转移函数 δ 的定义如下

$$\delta(s, \sigma) = \{t_1, t_2, \dots\} \subseteq S$$

- 类比量子力学的叠加态
 - 只要有一个可能的状态是接收态, 那就被认为接受

确定性 vs 非确定性

确定性(Deterministic)

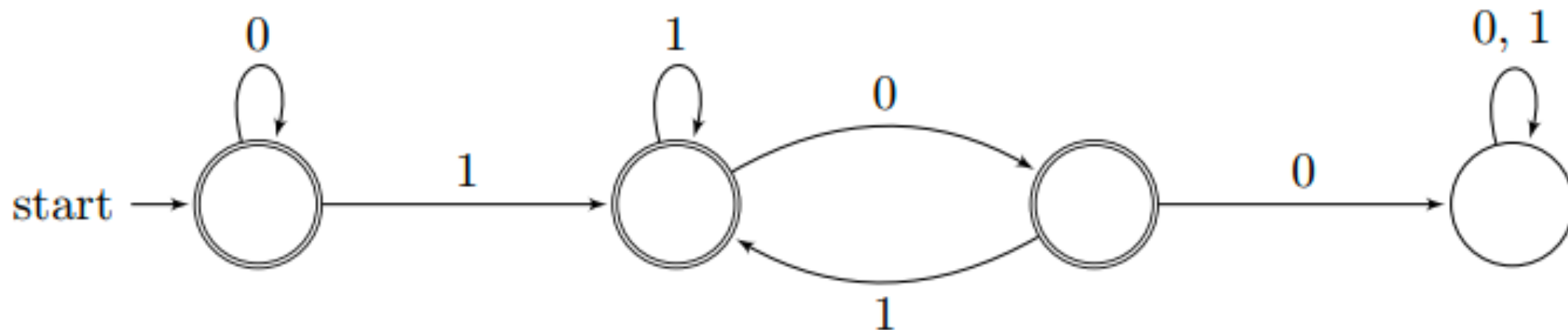
- 状态和输入确定唯一的转移
- 算法实现很简单
- 不够powerful

非确定性(Nondeterministic)

- 一个原始状态和一个输入会有多个转移状态
- 同一输入对应多种转移路径
- 只要有一个转移路径结果为AC态，结果即为AC

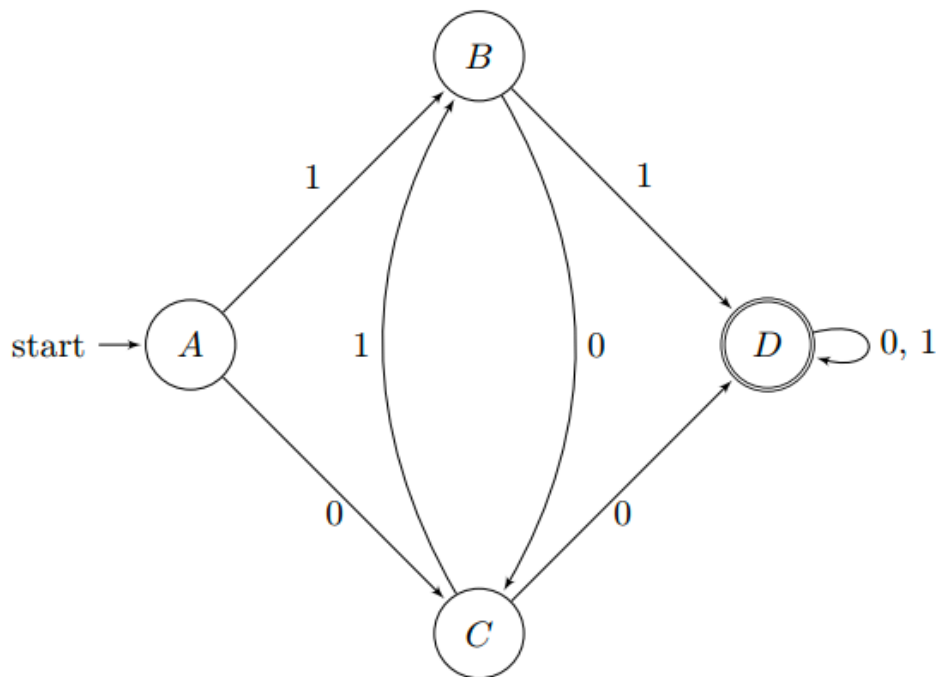
课堂练习

- 设计一个有限状态自动机，接收所有不含“100”字串的01字符串
 - 确定性
 - 非确定性？



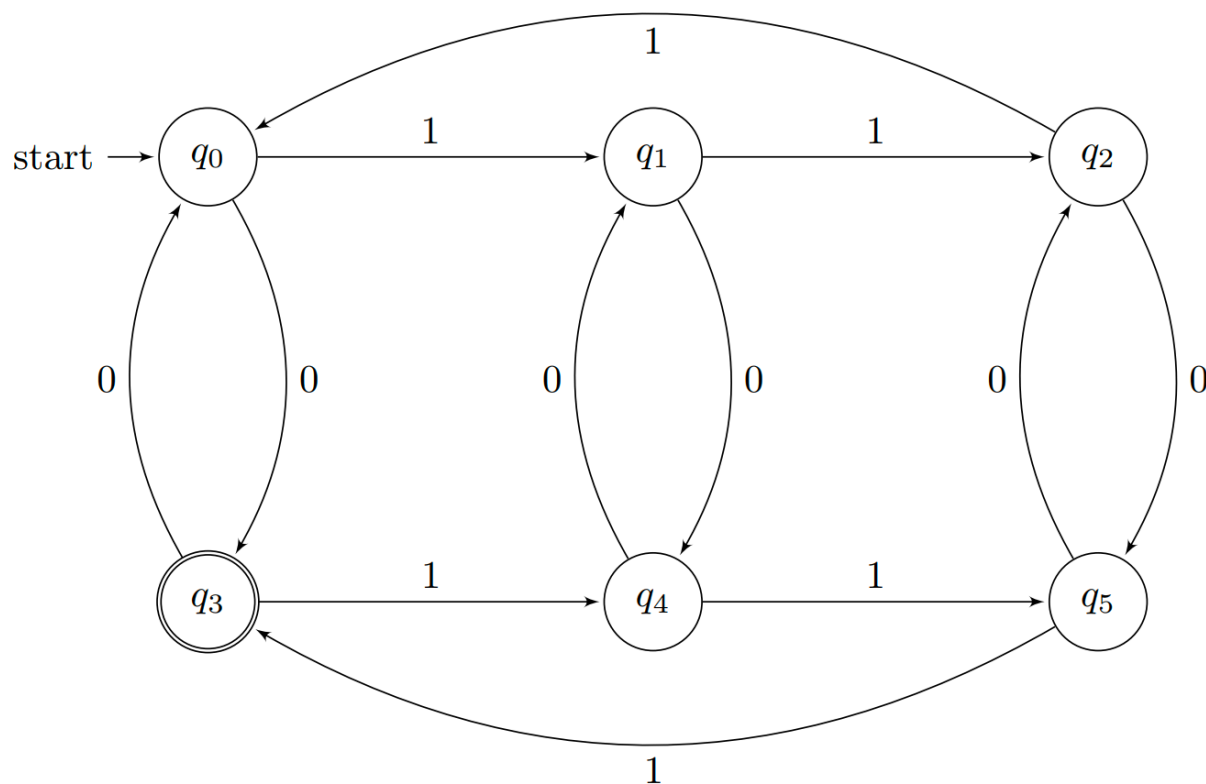
课堂练习

- 构造一个FSA，使其接收01字符串中包含两个连续0或两个连续1的串



课堂练习

- 构造一个FSA，使其能够接收包含奇数个0且其中1个数量可以被3整除的字符串
 - 0, 0111, 00



课堂练习

- 极具挑战版
 - 构造一个FSA，使其接收01字符串中有某些长度超过5的字串出现过两次的那些。两次出现可以有重叠
 - 000000
 - 0000100001
 - 100010001

DFA, NFA, 正则表达式的转换

- RE->NFA
- DFA->NFA
- NFA->DFA?